

GitClear — User Guide

GitClear reclaims disk space by finding the files Git ignores in your repositories and moves them to the Recycle Bin. It needs Git installed and on your `PATH`, because it asks **Git itself** what is ignored: what GitClear offers to delete is exactly what Git already considers disposable, so your tracked source code is never a candidate.

Clearing a repository

1. **Browse...** to a folder holding your repositories. GitClear searches it immediately and lists what it finds under **Repositories**.
 2. **Click a repository**. It is scanned straight away. One marked (**worktree**) is a Git worktree or submodule; it behaves the same.
 3. **Tick what you want cleared** in the **Folders** tree or the **Files** list.
 4. **Delete selected** → **Recycle Bin**, and confirm.
 5. **Undo last delete** puts it all back if you change your mind.
-

Why some folders behave differently

Some folders hold nothing but disposable files. Others hold work you care about with a few disposable files mixed in. GitClear can tell which is which, and treats them differently. In the **Folders** tree, a yellow folder icon (📁) beside the name marks the difference — hovering it says *"Entire folder is ignored — deleted as a unit"*.

With the 📁 icon — everything in that folder is disposable. GitClear deletes the whole folder in one action. That is why clearing a huge folder is instant, the folder does not open up to show what is inside, and you cannot keep just some of it. It goes entirely, or not at all.

Without the 📁 icon — that folder also holds files you are working on. GitClear deletes only the individual disposable files, which are the ones listed on the right when you select the folder. The folder itself, and everything you are working on inside it, stays exactly where it is.

Sizes are 1024-based, with Windows-style labels (`bytes` , `KB` , `MB` , `GB`).

Undo

Undo last delete restores the last deletion to its original location. Its limits are worth knowing:

- **One level.** The last deletion only, not a history.
- **Forgotten** when you switch repositories, and when you close GitClear.
- **Best-effort.** If Windows cannot restore automatically, the items are still in the Recycle Bin — restore them from there.

Because every deletion goes to the Recycle Bin, you can always restore manually, long after Undo has been forgotten.

What GitClear will never delete

- **Anything Git does not report as ignored.** GitClear never decides for itself what is disposable. Git honours your `.gitignore` files (including nested ones that override their parents), `.git/info/exclude`, and your global excludes.
- **A tracked file.** Tracked files are not ignored, so they never appear in the tree and cannot be selected.
- **Your `.git` folder, history, or index.**
- **A whole folder, unless Git proves everything in it is disposable** — the folder icon above. Any folder holding work you care about is only ever touched one file at a time.

It also skips folders it cannot read, and will not follow symbolic links out of the tree you pointed it at.

Very large deletions

Windows cannot recycle an item bigger than the Recycle Bin's capacity — it can only delete such an item permanently. A large dependency or build folder can easily be several gigabytes, well past a default allowance.

GitClear always asks first. If something cannot be recycled, Windows shows its permanent-delete warning:

Are you sure you want to permanently delete this folder?

- **No backs out.** GitClear stops and reports that nothing was permanently deleted; anything already recycled is still restorable with **Undo**.
- **Yes destroys those items for good.** They are **not** recoverable, and **Undo cannot bring them back**.

This is the only prompt GitClear lets through mid-operation. To avoid it, raise the Recycle Bin's capacity (right-click it → *Properties*) or delete in smaller batches.

Troubleshooting

Most messages are very clear.

Git could not scan this repository. GitClear shows Git's own explanation after "*Git says:*", so you do not need to run Git yourself. On network or mapped drives this is often Git's `dubious ownership` complaint, and Git's message includes the command that fixes it.

Some items could not be deleted. Usually a file locked by another program — a build, an editor, a running app, an antivirus scan. Close it and try again.

Limitations

- Deleting the disposable files from a folder that also holds your work can leave the now-empty folder behind, because GitClear cannot assume the folder itself is disposable. It takes no meaningful space.

- GitClear does not remember the last folder you scanned.
- Scan time is dominated by measuring file sizes; very large ignored trees take a few seconds.